



Consider the state diagram in Figure 4 (25 Mark)
Fig. 4

A- Draw the algorithmic state machine (ASM) of this state machine. And how many Flip flop you would need for the implementation?

B- Write the VHDL code of state transition rules and output generation for the state machines in the following template

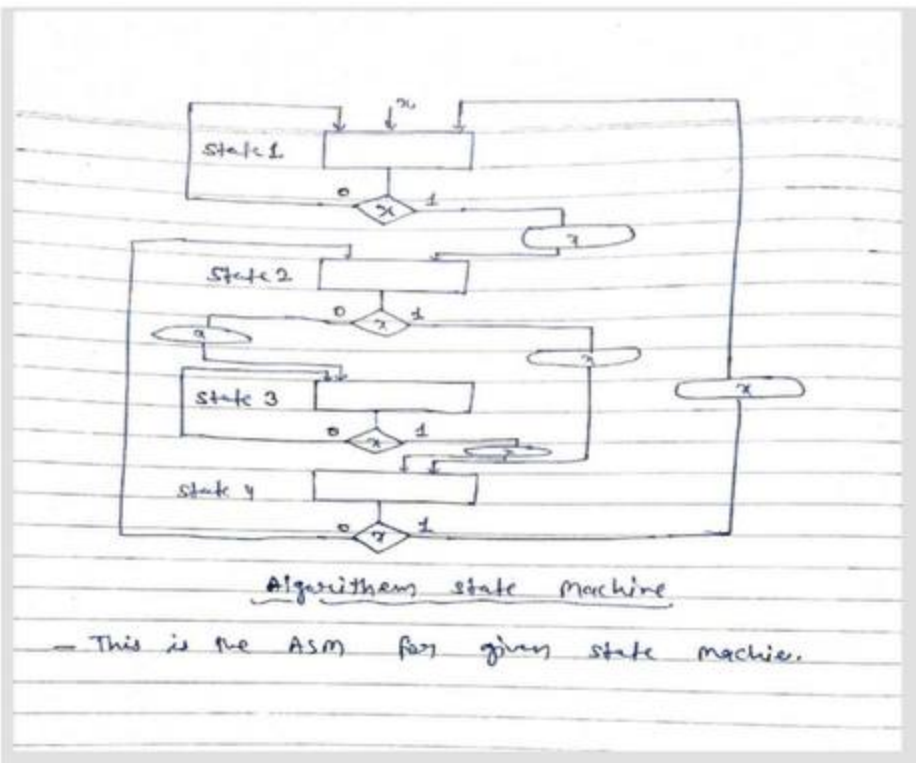
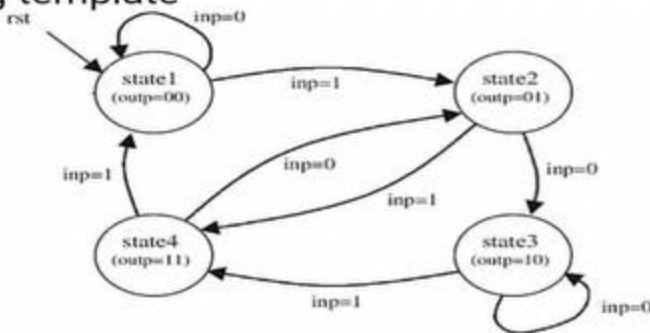




Fig. 2

Q2- Consider the Shift Register with Load shown in Figure 3. When load = '0', it operates as a regular shift register. However, if load = '1', din is loaded into the DFFs at the next positive clock transition (thus the initial state of the flip-flops can be programmed). The entity shown in dashed box has the interfaces as follows:

```
ENTITY MUXED_FF IS
  PORT (clk, din, d, load : IN STD_LOGIC;
        q : OUT STD_LOGIC);
END ENTITY;
```

Assuming you have this entity in your project do the following:

- 1) Design this circuit (4 stages) using VHDL using **GENERATE** directive.
- 2) Modify the design so that the number of stages is a **GENERIC** named stages

1)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
entity shift_register is
  port(clk,d,load : IN STD_LOGIC;
        din : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
        q : OUT STD_LOGIC_VECTOR(3 DOWNTO 0));
end entity;
```

```
architecture behave of shift_register is
  component MUXED_FF
    port(clk,din,d,load : IN STD_LOGIC;
          q : OUT STD_LOGIC);
  end component;
  Reg : MUXED_FF port map(.clk(clk),din(din(0)),d(d),load(load),q(q(0)))
begin
  GEN_REG:
  for i in 1 to 3 generate
    REGX : MUXED_FF port map
      (.clk(clk),din(din(i)),d(q(i-1)),load(load),q(q(i)));
  end generate GEN_REG;
end behave;
```

2)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

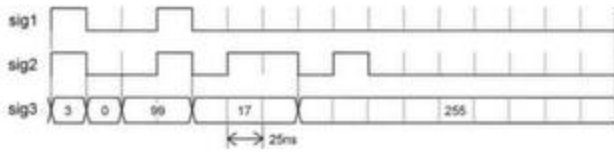
```
entity shift_register is
  generic(n : integer);
  port(clk,d,load : IN STD_LOGIC;
        din : IN STD_LOGIC_VECTOR(n-1 DOWNTO 0);
        q : OUT STD_LOGIC_VECTOR(n-1 DOWNTO 0));
end entity;
```

```
architecture behave of shift_register is
  component MUXED_FF
    port(clk,din,d,load : IN STD_LOGIC;
          q : OUT STD_LOGIC);
  end component;
  Reg : MUXED_FF port map(.clk(clk),din(din(0)),d(d),load(load),q(q(0)))
begin
  GEN_REG:
  for i in 1 to n-1 generate
    REGX : MUXED_FF port map
      (.clk(clk),din(din(i)),d(q(i-1)),load(load),q(q(i)));
  end generate GEN_REG;
end behave;
```





Q2: Write the testbench code to generate the signals in the figure below: (12 marks)



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity tb_latch is
end tb_latch;
```

```
architecture Behavioral of tb_latch is
```

```
signal sig1,sig2:std_logic;
```

```
signal sig3:std_logic_vector(11 downto 0);
```

```
begin
```

```
sig1<= '1', '0' after 25ns , '1' after 75ns , '0' after 100ns;
```

```
sig2<= '1', '0' after 25ns , '1' after 75ns , '0' after 100ns , '1' after 125ns , '0' after 175ns , '1' after 200ns , '0' after 225ns;
```

```
sig3<= "000000000011", "000000000000" after 25ns , "000010011001" after 50ns ,
```

```
"000000010111" after 75ns , "001001010101" after 150ns;
```

```
end Behavioral;
```

Q / Write vhdl code for a zero to nine counter with reset signal input and the output (state value) is displayed using 7-segment displays HEX0.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;
```

```
entity counter is
```

```
Port ( clk,rst : in STD_LOGIC;
```

```
out1 : inout STD_LOGIC_VECTOR (3 downto 0);
```

```
HEX0: out std_logic_vector(6 downto 0));
```

```
end counter;
```

```
architecture Behavioral of counter is
```

```
signal count:std_logic_vector(3 downto 0);
```

```
begin
```

```
process(clk,rst,count)
```

```
begin
```

```
if rst='1' then
```

```
count<="0000";
```

```
elsif rising_edge(clk) then
```

```
count<=count+'1';
```

```
end if;
```

```
out1<=count;
```

```
case out1 is
```

```
when "0000"=> HEX0<="1111110";
```

```
when "0001"=> HEX0<="0110000";
```

```
when "0010"=> HEX0<="1101101";
```

```
when "0011"=> HEX0<="1111001";
```

```
when "0100"=> HEX0<="0110011";
```

```
when "0101"=> HEX0<="1011011";
```

```
when "0110"=> HEX0<="1011111";
```

```
when "0111"=> HEX0<="1110000";
```

```
when "1000"=> HEX0<="1111111";
```

```
when "1001"=> HEX0<="1111011";
```

```
when others => HEX0<="0000000";
```

```
end case;
```

```
end process;
```

```
end Behavioral;
```



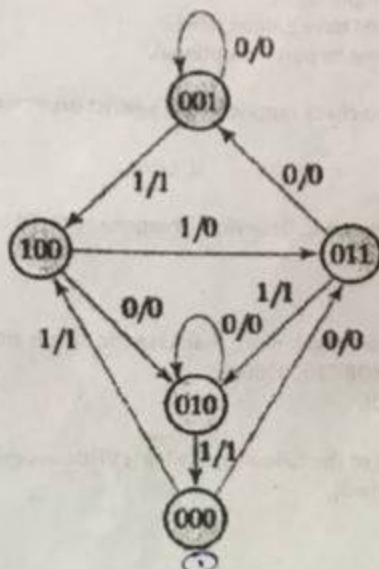
END PROCESS;
END ARCHITECTURE;

Q2: Consider the state diagram in Figure Q2

Department of Electronic and Communications Engineering		Date: 8/8/2021 Examiner: Dr. Ammar Jallawi Mahmood
2 nd Semester 2019-2020, Final Exam, 1 st Attempt	Note: Answer all questions	

Q1: Consider the state diagram:

- A- Draw the ASM of this FSM assuming the initial state is "000" (10 Marks)
B- write the VHDL code for a proper implementation of this FSM described by this state diagram using D flip-flops?



Q2: To test a 3x8 Decoder using Records, complete the following record declaration and its constant of array: (5 Marks)

type test_vector is record

d : std_logic_vector(.....);

o : std_logic_vector(.....);

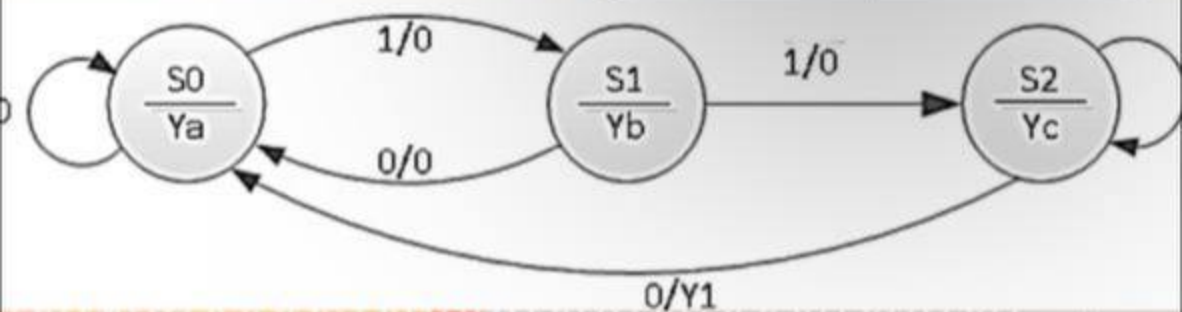
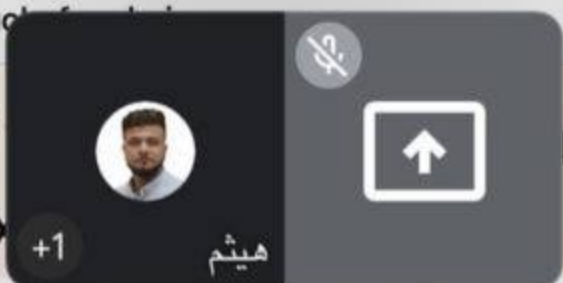
end record;

type test_vector_array is array of test_vector;

constant test_vectors : test_vector_array := (

(.....),
(.....),
(.....),
(.....),
(.....),
(.....),
(.....);

be "0" or "0" of the decision box



Machine to ASM Example

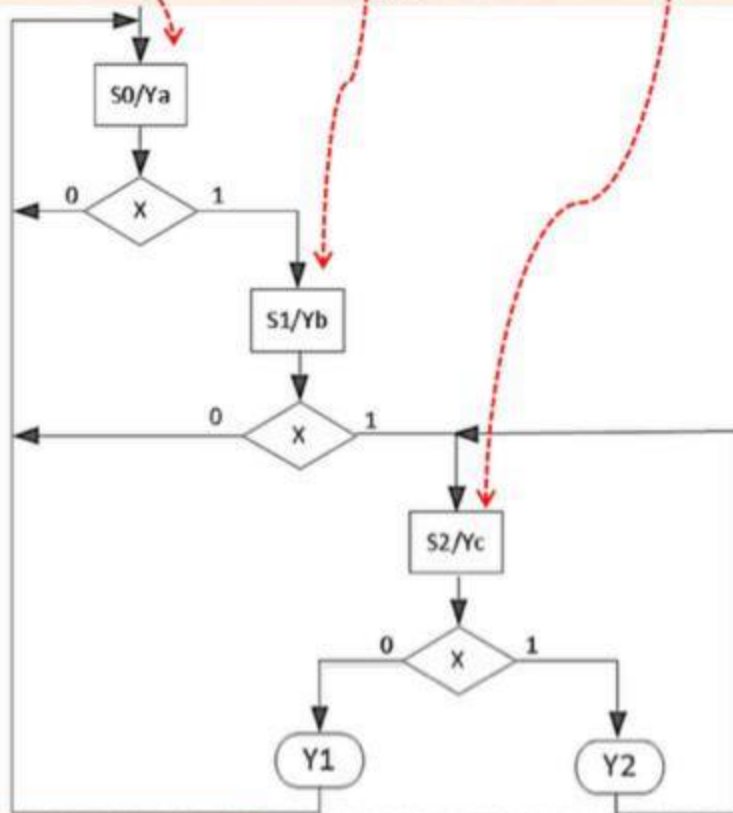
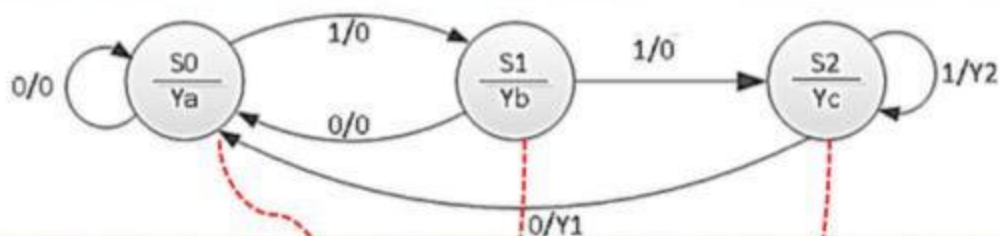


Chart to VHDL

the **ASM chart** is determined, the version to *HDL* is straight forward.

use statement can be used to specify opens in each state

Date: 8/8/2021
Examiner:
Ammar Jallawi Mahmoud

(10 Marks)
this state diagram using D

Design this circuit for 4 stages using VHDL (10 Marks)
The entity shown in dashed box has the interfaces as follows:

```
ENTITY MUXED_FF IS
  PORT (clk, D, S, Load : IN STD_LOGIC;
        Q : OUT STD_LOGIC);
END ENTITY;
```

Assume you have this entity in your library. Write VHDL to design the same circuit assuming the number of stages is a GENERIC named N and using GENERATE statement. (10 Marks)

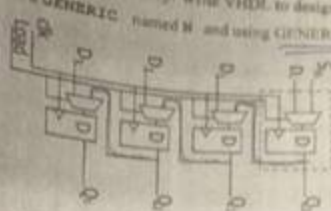


Figure Q3

Write a complete VHDL code

```
Entity MUXED_FF is
  Port (clk, D, S, load : in std_logic;
        Q : out std_logic);
end entity;
```

```
entity shr is
  Port (clk, D, S, load : in std_logic;
        Q : out std_logic (3 to 0));
end entity;
```

architecture str of Shr is

signal D : std_logic_vector (3 downto 0)

begin

```
Shr_loop: for i in 0 to 3 generate
```

```
Shr: Max entity work, MUXED_FF
```

```
Portmap (clk=>clk, D=>D(i), Load=>load, S=>S, Q=>Q(i))
```

```
Q(i) <= D(i+1)
```


Design this circuit for 4 stages using VHDL (10 Marks)
The entity shown in dashed box has the interfaces as follows:

```
ENTITY MUXED_FF IS
  PORT (clk, D, S, Load : IN STD_LOGIC;
        Q : OUT STD_LOGIC);
END ENTITY;
```

Assume you have this entity in your library. Write VHDL to design the same circuit assuming the number of stages is a **GENERIC** named **N** and using **GENERATE** statement. (10 Marks)

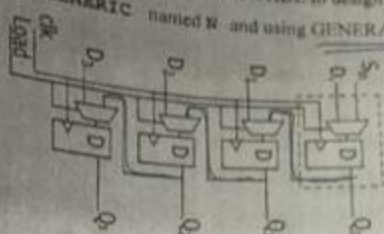


Figure Q3

Write a complete testbench VHDL code of the processes that generate the signals A, B, C, D and E of figure Q4. Employ **WAIT FOR** statement. Each stimuli should be generated in a separate process. Note that A and B are periodic signals and should be generated indefinitely (10 Marks)

```
Entity MUXED-FF is
  Port (clk, D, S, load : in std_logic;
        Q : out std_logic);
end entity;
```

```
entity shr is
  generic (N : integer := 4);
  Port (clk, D, S, load : in std_logic;
        Q : out std_logic (N-1));
end entity;
```

```
architecture str of shr is
  signal D : std_logic_vector (N-1 downto 0);
```

```
begin
  N-1
  shr loop: for i in 0 to N-1 generate
```

```
shr: Max entity work, MUXED-ff
```

```
portmap (clk=>clk, D=>D(i), Load=>load, S=>S, Q=>Q(i));
```

```
Q(i) <= d(i) d(N)
```

plement a 4x1 multiplexer with:

A- where inputs are single bit and using only logical operators (gate level)

B- N-bit inputs specify N using GENERIC and using WHEN statement

... the signals in the figure bellow:

%٨٣ |||

• ٤:٣٩

Answer

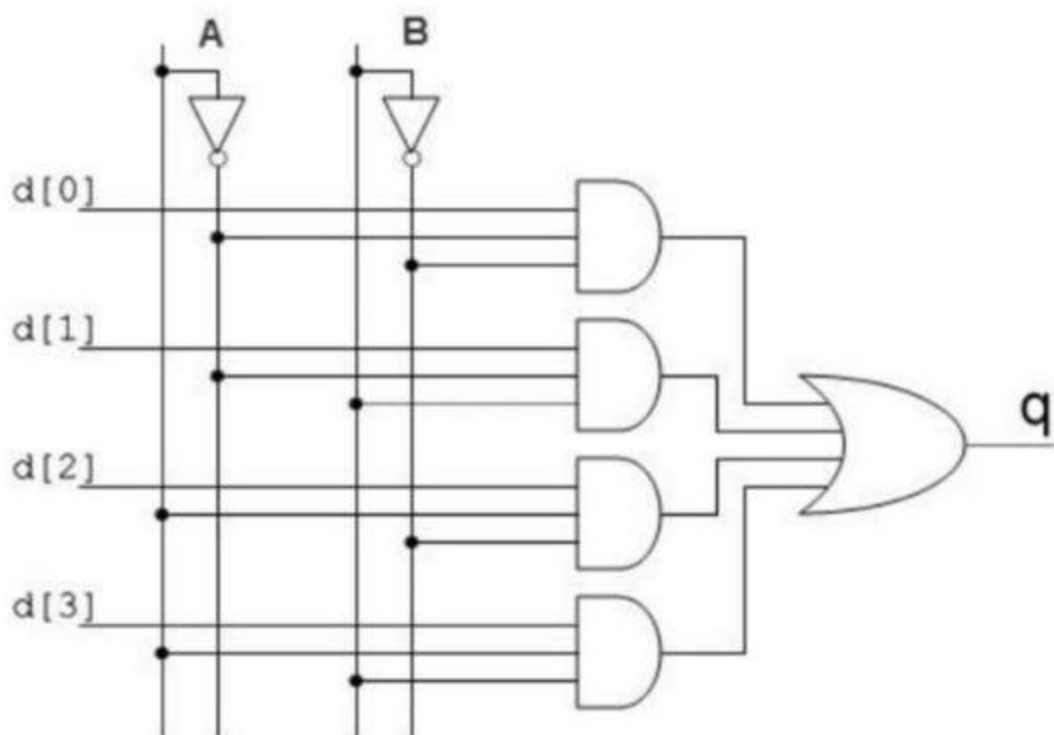
Q-2)

A)

Answer:

4 x 1 mux need 4 AND and 1 OR and 2 NOT gates.

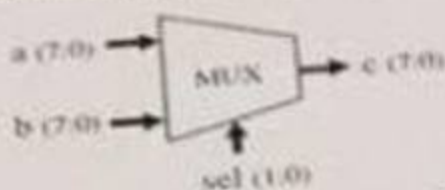
Replace OR gate with Invert-input NAND , then above circuit will be replace with 2 NOT and 5 NAND , then 2 NANDS for 2 NOT's , then total 7 NAND gates required.



Reviews: Likes: 0 Dislikes: 0

1/2

p-level diagram of a multiplexer is shown in the figure 1. According to the truth table, the output should be equal to one of the inputs if sel = "01" (c = a) or sel = "10" (c = b), but it should be equal to 'Z' (high impedance) if sel = "00" or sel = "11", respectively.



sel	c
00	0
01	a
10	b
11	Z

Fig. 1

Write the VHDL code below and write relevant comments regarding your solution. The VHDL code to implement the same circuit using **WHEN** or **CASE**. The code so that the input vector size is **GENERIC** name **d_width**.

```

1  -----
2  LIBRARY ieee;
3  USE ieee.std_logic_1164.all;
4  -----
5  ENTITY mux IS
6      PORT ( a, b : STD_LOGIC_VECTOR (7 DOWNTO 0);
7             sel : IN STD_LOGIC_VECTOR (1 DOWNTO 0);
8             c : OUT STD_LOGIC_VECTOR (7 DOWNTO 0));
9  END mux;
10 -----
11 ARCHITECTURE example OF mux IS
12 BEGIN
13     PROCESS (a, b, sel)
14     BEGIN
15         IF (sel = "00") THEN
16             c <= "00000000";
17         ELSIF (sel = "01") THEN
18             c <= a;
19         ELSIF (sel = "10") THEN
20             c <= b;
21         ELSE
22             c <= (OTHERS => 'Z');
23         END IF;
24     END PROCESS;
25 END example;
26 -----

```

Fig. 2

Under the Shift Register with Load shown in Figure 3. When load = '1', the data '1' is loaded into the DFFs at the next clock edge. The entity shown in Figure 3 is:

This problem has been solved!

See the answer



100% 3G

9:07

```

2. LIBRARY IEEE;
3. USE IEEE.std_logic_1164.all;
4. -----
5. ENTITY mux IS
6.     PORT (a,b : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
7.           sel : IN STD_LOGIC_VECTOR (1 DOWNTO 0);
8.           x : OUT STD_LOGIC_VECTOR (7 DOWNTO 0));
9. END mux;
10. -----
11. ARCHITECTURE example OF mux IS
12. BEGIN
13.     PROCESS (a,b,sel)
14.     BEGIN
15.         IF (sel="00") THEN
16.             x <= "00000000";
17.         ELSEIF (sel="01") THEN
18.             x <= a;
19.         ELSEIF (sel="10") THEN
20.             x <= b;
21.         ELSE
22.             x <= "00000000";
23.         END IF;
24.     END PROCESS;
25. END example;

```

Q1

(b.) We can use when or if statements for designing a mux as they will also work same as of case statements. Here the code will get triggered at each and every change in the inputs "a" and "b" or at any change in selection line "sel", as these three are in the sensitivity list.



3G
"1001"

hence the correct statement to enter will be digit_flipper. ۳۲

Reviews: Likes: 0 Dislikes: 0

Q4: Write the code for a zero to nine counter with reset signal input and the output (state value) is displayed using 7-segment displays HEX0. (15 m)

$$\begin{array}{r} 15 \\ \times 10 \\ \hline 150 \end{array}$$

Dr. Ammar Jallawi Mahmood

```
library ieee;
use ieee.std_logic_1164.all;
entity counter is
    port ( clk, reset : in std_logic;
          display : out std_logic_vector (6 downto 0));
end counter;
```

Architecture Counter of counter is
begin

process (clk, reset)

variable temp : integer range 0 to 10

begin

if (reset = '1') then

temp = 0

elsif (clk = 'even' and clk = '1') then

temp = temp + 1

if (temp = 10) then

temp = 0;

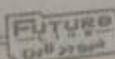
end if

end if

Q4

Case temp is

when 0 $\Rightarrow$ display $\leftarrow$ 000 111 111 0;



- ☒ a. $Y \leftarrow \text{to_unsigned}(35, 4);$
☐ b. $X \leftarrow \text{unsigned}(Y);$
☐ c. $X \leftarrow \text{to_integer}(Y);$
☒ d. $Y(5 \text{ downto } 0) \leftarrow \text{to_unsigned}(17, 6);$
☒ e. $Z \leftarrow \text{to_integer}(\text{unsigned}(x));$

Q5: Determine which lines have syntax errors in the accompanying VHDL code and write the corrected code (5 Marks)
 Note selecting wrong lines would cause a penalty:

1 library ieee;
 2 use ieee.std_logic_1164.all;
 3 entity Error_finder is
 4 generic
 5 (NUM_STAGES : natural := 16);
 6 port
 7 (clk, enable, reset, sr_in: std_logic;
 8 sr_out : out std_logic);
 9 end entity;
 10 architecture rtl of Error_finder is
 11 type sr_length is array ((NUM_STAGES-1) to 0) of std_logic;
 12 signal sr: sr_length;
 13 begin
 14 process (clk, reset)
 15 begin
 16 if (reset = '1') then
 17 sr <= (others => '0');
 18 elsif (rising_edge(clk)) then
 19 if (enable = '1') then
 20 sr((NUM_STAGES-1) downto 1) <= sr((NUM_STAGES-2) downto 0);
 21 sr(0) <= sr_in;
 22 end if;
 23 end process;
 24 sr_out <= sr(NUM_STAGES-1 downto 0);
 25 end rtl;

Good luck although you shouldn't need it!

Answer

Syntax errors are in line 7 and 9.

In the line 9 it must be end Error_finder;

In the line 7 syntax must be individual things not separated by commas.



```
x(3) <= x(5 DOWNT0 5);
```

```
-- F. Different types. x(5 DOWNT0 5)
```

```
-- x(3) STD_LOGIC.
```

```
-- x(5 DOWNT0 5) STD_LOGIC_VECTOR.
```

```
-- F. Different types.
```

```
b <= x(5 DOWNT0 5);
```

Q2- Consider the parallel adder shown in Figure 3. Write VHDL to generate an N bit adder from a full adder. N should be defined as generic. The entity shown in dashed box (fulladder) has the interfaces as follows:

```
entity full_adder is
    port (x,y,Cin: in std_logic;
          S, Cout: out std_logic);
end full_adder;
```

Assuming you have this entity in your project do the following:

- 1) Design this circuit (8 stages) using VHDL using **GENERATE** directive. (5 Marks)

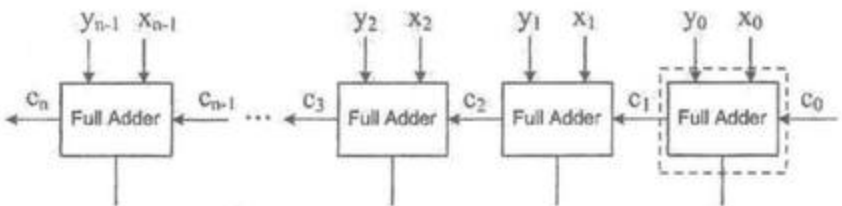


Figure Q2

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY StageAdder IS
    PORT (
        C0: IN std_logic;
        x: IN std_logic_vector(7 DOWNT0 0);
        y: IN std_logic_vector(7 DOWNT0 0);
        Cn: OUT std_logic;
        s: OUT std_logic_vector(7 DOWNT0 0));
END ENTITY;

ARCHITECTURE StageAdder OF StageAdder IS
    SIGNAL C : std_logic_vector(8 DOWNT0 0);
BEGIN
    C(0) <= C0;
    Cn <= C(8);

    m1:FOR i IN 0 TO 7 GENERATE
        L1: ENTITY work.full_adder
            PORT MAP ( x => x(i),
                      y => y(i),
                      Cin => C(i),
                      S => s(i),
                      Cout => C(i+1));
    END GENERATE;
END ARCHITECTURE;
```





3 من 12

```

m1:FOR i IN 0 TO 7 GENERATE
  L1: ENTITY work.full_adder
    PORT MAP ( x => x(i),
               y => y(i),
               Cin => C(i),
               S => s(i),
               Cout => C(i+1));

  END GENERATE;
END ARCHITECTURE;

```

2) Modify the design so that the number of stages is a **GENERIC** named N. (5 Mar

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY NStageAdder IS
  GENERIC (N: integer:= 8);
  PORT (
    C0: IN std_logic;
    x: IN std_logic_vector(N - 1 DOWNT0 0);
    y: IN std_logic_vector(N - 1 DOWNT0 0);
    Cn: OUT std_logic;
    s: OUT std_logic_vector(N - 1 DOWNT0 0));
END ENTITY;

ARCHITECTURE StageAdder OF StageAdder IS
  SIGNAL C : std_logic_vector(N DOWNT0 0);
BEGIN
  C(0) <= C0;
  Cn <= C(N);

  m1:FOR i IN 0 TO N - 1 GENERATE
    L1: ENTITY work.full_adder
      PORT MAP ( x => x(i),
                 y => y(i),
                 Cin => C(i),
                 S => s(i),
                 Cout => C(i + 1));

    END GENERATE;
END ARCHITECTURE;

```





3) ANSWER (A) OR (B)

- (A) A fundamental synthesizable value is 'Z' (high impedance), which is needed to create tri-state buffers, like that depicted in Figure Q3 (see its truth table). Write a VHDL code from which this circuit can be inferred. (10 Marks)

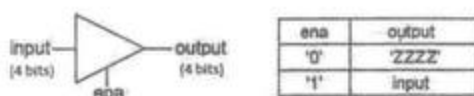


Figure Q3

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY buffer1 IS
PORT (
    input: IN std_logic_vector(3 DOWNTO 0);
    ena: IN std_logic;
    output: OUT std_logic_vector(3 DOWNTO 0));
END ENTITY;

ARCHITECTURE buffer1 OF buffer1 IS
BEGIN
    Output <= input WHEN ena = '1' ELSE
        "ZZZZ";
END ARCHITECTURE;
```





(B) Examine the code below and answer the following:

- 1) Describe the circuit this code is inferring and draw the circuit diagram. (5 Marks)
- 2) Rewrite the code using `STD_LOGIC_VECTOR` represent the signals `q0 – q3`. (5 Marks)

```

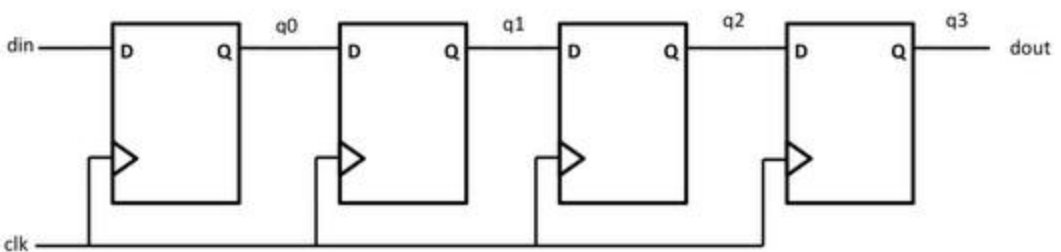
-----
ENTITY test IS
PORT (din, clk: IN STD_LOGIC;
      dout: OUT STD_LOGIC);
END ENTITY;
-----

ARCHITECTURE circl OF test IS
    SIGNAL q0, q1, q2, q3: STD_LOGIC;
BEGIN
    PROCESS (clk)
    BEGIN
        IF clk'EVENT AND clk='1' THEN
            q0 <= din;
            q1 <= q0;
            q2 <= q1;
            q3 <= q2;
        END IF;
    END PROCESS;
    dout <= q3;
END ARCHITECTURE;
-----

```

1)

Delays the input 4 clock cycles.



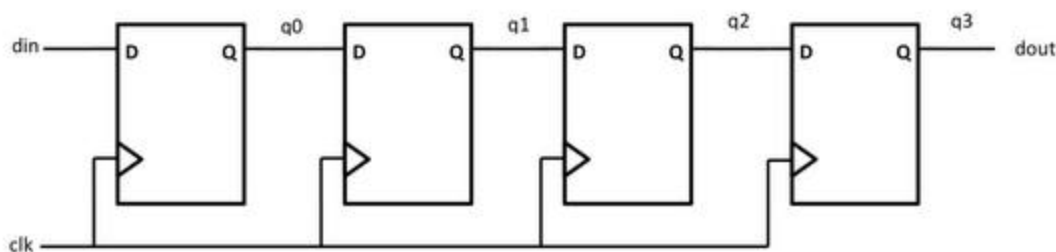


END ARCHITECTURE;

1)

6 من 12

Delays the input 4 clock cycles.



2)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY test IS
PORT (
    din, clk: IN std_logic;
    dout: OUT std_logic);
END ENTITY;

ARCHITECTURE circl OF test IS
    SIGNAL q: std_logic_vector(3 downto 0);
BEGIN
    PROCESS(clk)
    BEGIN
        IF (rising_edge(clk)) THEN
            q(0) <= din;
            q(1) <= q(0);
            q(2) <= q(1);
            q(3) <= q(2);
        END IF;
    END PROCESS;

    dout <= q(3);
END ARCHITECTURE;

--L1: FOR i IN 0 TO 2 LOOP
--q(i + 1) <= q(i);
--END LOOP;
```





Q4- Consider a circuit name DUT which has the following Entity definition:

```
ENTITY DUT IS
PORT {clk, rst, x, y: IN STD_LOGIC;
      z: IN STD_LOGIC_VECTOR (7 downto 0);
      Q: OUT STD_LOGIC_VECTOR (7 downto 0)};
END ENTITY;
```

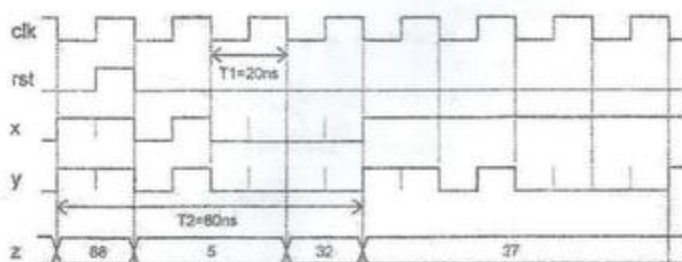


Figure Q4

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY test IS
END ENTITY;

ARCHITECTURE test OF test IS
    SIGNAL clk, rst, x, y: std_logic;
    SIGNAL z, Q: std_logic_vector(7 DOWNTO 0);
BEGIN
    DUT: ENTITY work.dut
    PORT MAP(clk => clk, rst => rst, x => x, y => y, z => z, Q => Q);

    PROCESS
    BEGIN
        clk <= '0';
        WAIT FOR 10 ns;
        clk <= '1';
        WAIT FOR 10 ns;
    END PROCESS;

    PROCESS
    BEGIN
        rst <= '0';
        WAIT FOR 10 ns;
        rst <= '1';
        WAIT FOR 10 ns;
        rst <= '0';
        WAIT;
    END PROCESS;
```

```
PROCESS
BEGIN
```





```

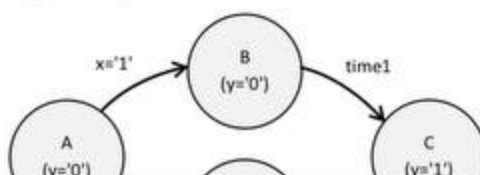
PROCESS
BEGIN
    x <= '1';
    WAIT FOR 20 ns;
    x <= '0';
    WAIT FOR 10 ns;
    x <= '1';
    WAIT FOR 10 ns;
    x <= '0';
    WAIT FOR 40 ns;
    x <= '1';
    WAIT;
END PROCESS;

PROCESS
BEGIN
    y <= '1';
    WAIT FOR 20 ns;
    y <= '0';
    WAIT FOR 10 ns;
    y <= '1';
    WAIT FOR 10 ns;
    y <= '0';
    WAIT FOR 40 ns;
    y <= '1';
    WAIT FOR 20 ns;
    y <= '0';
    WAIT FOR 10 ns;
    y <= '1';
    WAIT FOR 10 ns;
    y <= '0';
    WAIT FOR 40 ns;
    y <= '1';
    WAIT;
END PROCESS;

PROCESS
BEGIN
    z <= std_logic_vector(to_unsigned(88, 8));
    WAIT FOR 20 ns;
    z <= std_logic_vector(to_unsigned(5, 8));
    WAIT FOR 40 ns;
    z <= std_logic_vector(to_unsigned(32, 8));
    WAIT FOR 20 ns;
    z <= std_logic_vector(to_unsigned(27, 8));
    WAIT;
END PROCESS;
END ARCHITECTURE;

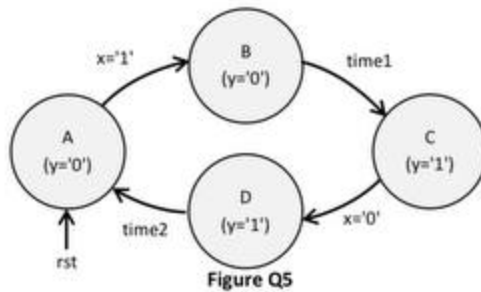
```

Q5- Consider the state diagram in Figure Q5

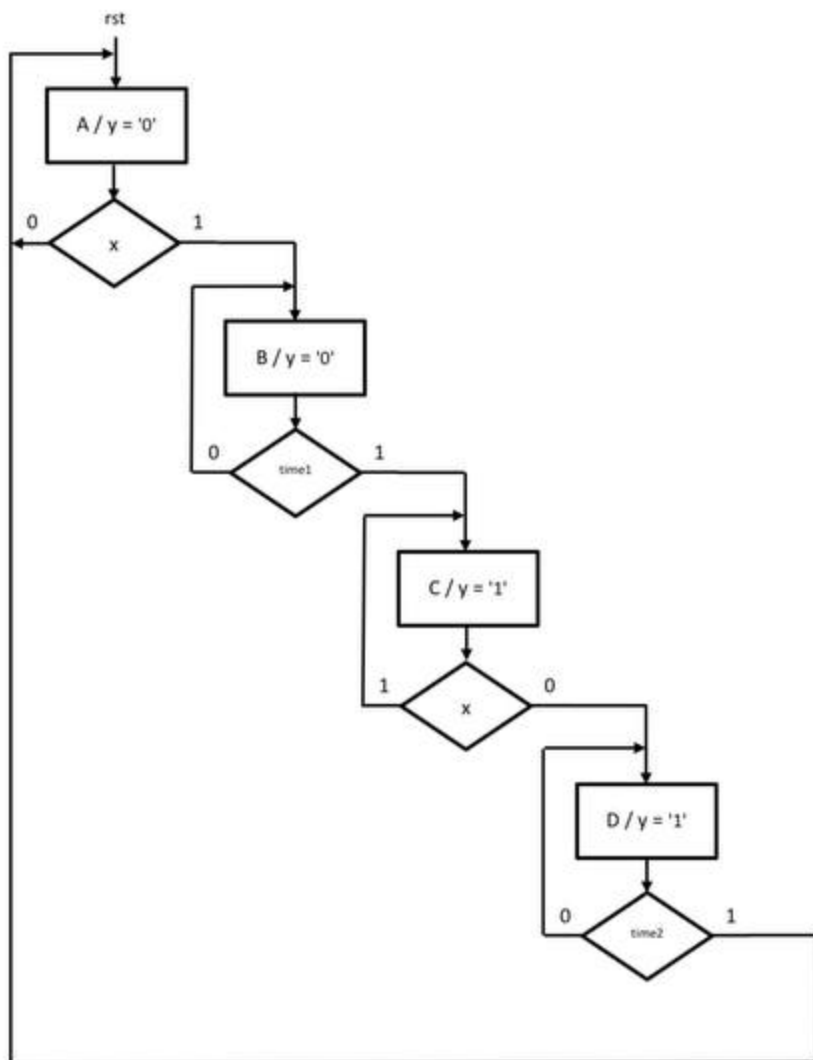




Q5- Consider the state diagram in Figure Q5



(A) Draw the algorithmic state machine (ASM) of this state machine. (5 Marks)



(B) Complete the VHDL code of state transition rules and output generation for the state machine in the following template. (5 Marks)

```

LIBRARY ieee;
USE ieee.stdlogic1164.all;
  
```





10 من 12

- (B) Complete the VHDL code of state transition rules and output generation for the state machine in the following template. (5 Marks)

```

LIBRARY ieee;
USE ieee.stdlogic1164.all;
ENTITY fsm IS
PORT ( x, time1, time2, clk, rst: IN STD_LOGIC;
      y: OUT STD_LOGIC_VECTOR);
END fsm;

ARCHITECTURE fsm OF fsm IS TYPE state IS (A, B, C, D);
SIGNAL present_state, next_state: state;
BEGIN
  PROCESS (rst, clk)
  BEGIN
    -- State transition implementation

  END PROCESS;

  PROCESS (x, time1, time2, present_state) BEGIN
    CASE present_state IS
      WHEN A =>
        -- State A implementation
      WHEN B =>
        -- State B implementation
      WHEN C =>
        -- State C implementation
      WHEN D =>
        -- State D implementation
    END CASE;
  END PROCESS;

  PROCESS (present_state)
  BEGIN
    -- Moore output computation (Y)
  END PROCESS;
END fsm;
  
```

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
  
```





```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY fsm IS
PORT (
    x, time1, time2, clk, rst: IN std_logic;
    y: OUT std_logic);
END ENTITY;

ARCHITECTURE fsm OF fsm IS
    TYPE state IS (A, B, C, D);
    SIGNAL present_state, next_state: state;
BEGIN
    PROCESS(clk)
    BEGIN
        IF(rising_edge(clk)) THEN
            IF(rst = '1') THEN
                present_state <= A;
            ELSE
                present_state <= next_state;
            END IF;
        END IF;
    END PROCESS;

    PROCESS(x, time1, time2, present_state)
    BEGIN
        CASE present_state IS
            WHEN A =>
                IF x = '1' THEN
                    next_state <= B;
                ELSE
                    next_state <= A;
                END IF;
            WHEN B =>
                IF time1 = '1' THEN
                    next_state <= C;
                ELSE
                    next_state <= B;
                END IF;
            WHEN C =>
                IF x = '0' THEN
                    next_state <= D;
                ELSE
                    next_state <= C;
                END IF;
            WHEN D =>
                IF time2 = '1' THEN
                    next_state <= A;
                ELSE
                    next_state <= D;
                END IF;
        END CASE;
    END PROCESS;

    y <= '0' WHEN present_state = A OR present_state = B ELSE
        '1';
END ARCHITECTURE;
```

